

IDS-NGNN: A SmartNIC-based Intrusion Detection System Based on Reduce and Merge Nested Graph Neural Networks

Rana Abu Bakar^{*†}, Francesco Paolucci[†], Filippo Cugini[†], Juan Jose Vegas Olmos[‡], and Lorenzo De Marinis^{*}

^{*}Scuola Superiore Sant'Anna, Pisa, Italy

Emails: {rana.abubakar, lorenzo.demarinis}@santannapisa.it

[†]CNIT, Pisa, Italy

[‡]Nvidia, Copenhagen, Denmark

Abstract—The growing complexity of network attacks has outpaced the capabilities of traditional intrusion detection systems (IDS), which often rely on flat data structures that fail to capture complex relationships within networks. To address this limitation, we propose IDS-NGNN, a novel IDS that integrates hardware-offload SmartNIC preprocessing with a nested graph neural network (NGNN) architecture. Unlike standard Graph Neural Networks (GNN), IDS-NGNN jointly captures local and global dependencies using a three-layer design: an internal GNN for host-level activity, a nested graph module for hierarchical aggregation, and an external GNN for inter-host communication. SmartNIC acceleration enables efficient real-time processing of large-scale graph-structured network data at the edge. We evaluate IDS-NGNN on six public IDS datasets, including CIC-IDS-2017, CSE-CIC-IDS-2018, and ToN-IoT. Experimental results demonstrate that IDS-NGNN achieves up to 95% accuracy and 92% F1-score, while maintaining efficiency suitable for real-time 100 Gbps deployments.

Index Terms—DDoS, Graph Neural Networks, Deep Learning, Networks, Intrusion Detection System

I. INTRODUCTION

The growing complexity and scale of cyberattacks pose a critical threat to organizations worldwide. Between 2020 and 2024, data breaches rose by 30% compared to 2019 [1], [2].

In 2024, Cloudflare reported blocking 21.3 million Distributed Denial-of-Service (DDoS) attacks, a 53% increase over 2023 [3], and a further 20.5 million in Q1 2025 alone, a 358% year-over-year surge, including 700 hyper-volumetric events above 1 Tbps and record floods of 4.8 billion packets/s [4]. Motivations can vary from financial gain to political disruption, highlighting the urgent need for effective security measures [5]. Intrusion Detection Systems (IDSs) are used to protect infrastructures by identifying and mitigating unauthorized access attempts [6].

Machine Learning (ML) and Deep Learning (DL) have enhanced IDS accuracy but face constant challenges. Inefficient feature extraction limits the capture of temporal-spatial attack patterns, especially in encrypted traffic [7], and computational overhead hinders real-time use in 100 to 400 Gbps networks [8]. Adversarial threats such as model poisoning reduce reliability [9]–[12]. Scalability remains constrained by dataset bias, domain adaptation, and hardware inefficiency [13], [14].

Graph-based representations have emerged as a promising alternative solution by modeling hosts as nodes and their interactions as edges [15]. GNN extends this approach by enabling expressive embeddings for security tasks such as malware detection, threat detection, and fraud analysis [12], [16]. However, most GNN-based IDS models still rely on flat message-passing architectures, which cannot jointly capture host-level and global dependencies, and suffer from over-smoothing in deeper layers. To address this, Zhang et al. [17] proposed the Nested Graph Neural Networks (NGNN), introducing hierarchical subgraphs to mitigate over-smoothing and improve expressivity. But its potential in network security remains unexplored.

Several GNN-based IDS frameworks have been proposed, including E-GraphSAGE [18] and PPT-GNN [19], achieving strong results on datasets such as CIC-IDS2017 and UNSW-NB15. However, challenges remain, such as optimal graph construction for network flows remaining underexplored, deep GNNs incurring computational inefficiencies, and robustness against structural adversarial attacks is limited.

To address these issues, we propose **IDS-NGNN**, a novel IDS based on a nested GNN architecture tailored for large-scale network intrusion detection. Unlike flat GNNs, IDS-NGNN employs a three-layer hierarchy that separates fine-grained host behaviors from global inter-host interactions, enabling robust detection of distributed attacks. By integrating SmartNIC-based flow preprocessing with GPU-accelerated training, IDS-NGNN achieves high-speed, low-latency inference suitable for real-world deployment. Evaluations across six public datasets show IDS-NGNN consistently improves accuracy, scalability, and robustness under adversarial perturbations, while fulfilling the real-time constraints of high-speed networks.

A. Motivation for IDS-NGNN Framework

Conventional IDS models and recent GNN-based approaches rely on flat GNN structures that uniformly aggregate neighbor features [18]–[20]. However, flat GNNs are limited by over-smoothing and lack hierarchical context. IDS-NGNN addresses these issues with a three-layer nested design that detects coordinated threats such as lateral movement and DDoS attacks.

Proposition 1. Flat GNNs of depth k suffer from over-smoothing once k exceeds the mixing time of the underlying graph, causing embeddings to converge to indistinguishable values [21], [22]. IDS-NGNN avoids this by decomposing the representation into internal, nested, and external layers, each with bounded propagation depth.

Justification. The internal layer restricts propagation to 1–2 hops within host-centric subgraphs, ensuring fine-grained local capture. The nested layer compresses embeddings into compact host-level descriptors, reducing redundancy and limiting adversarial impact. The external layer propagates once across the inter-host graph, efficiently capturing global dependencies. Together, this design maintains hierarchical expressivity, avoids embedding collapse, and improves robustness against structural perturbations [23]. Complexity remains tractable: $O(E_{in}d^2)$ for internal GNNs, $O(Nd)$ for the nested layer, and $O(E_{out}d^2)$ for the external GNN, where E_{in} and E_{out} are internal and external edges, N is the number of hosts, and d the embedding size. Combined with SmartNIC preprocessing and GPU acceleration, IDS-NGNN meets the real-time requirements of high-speed networks.

II. PROPOSED IDS-NGNN FRAMEWORK

In this section, we introduce the **IDS-NGNN** framework to address the limitations of traditional GNNs in network intrusion detection. IDS-NGNN captures both local and global relational patterns within network traffic. The core idea of IDS-NGNN is constructing a two-tier nested graph structure as an **internal graph** for each host, capturing its localized communication behavior and internal activity. As an **external graph**, constructed over hosts, capturing inter-host communication patterns.

Algorithm 1 IDS-NGNN Forward Pass

Require: Graph batch D with nodes x , edge indices E , labels y , and batch vector b

Ensure: Node predictions O , internal embeddings I , external embeddings E_{ext} , graph-level embedding G

- 1: $I \leftarrow \text{InternalGATLayer}(x, E)$ {Compute internal node embeddings using GAT}
 - 2: $N \leftarrow \text{NestedGraphLayer}(I, E)$ {Aggregate internal embeddings (map-reduce step)}
 - 3: $E_{ext} \leftarrow \text{ExternalGATLayer}(N, E)$ {Compute external node embeddings using GAT}
 - 4: $G \leftarrow \text{GlobalMeanPool}(E_{ext}, b)$ {Aggregate node embeddings to obtain a graph-level embedding}
 - 5: $O \leftarrow \text{Classifier}(E_{ext})$ {Predict node labels (normal vs. compromised)}
 - 6: **return** O, I, E_{ext}, G
-

This hierarchical design enables the model to continuously analyze fine-grained subgraph features and higher-level inter-host interactions. The three layers of IDS-NGNN are described in Figure 1. Internal GNN applies a Graph Attention Network (GAT)-based message passing operation over each internal graph to learn node embeddings reflecting intra-host behavior. The Nested Graph Layer aggregates internal graph embeddings

and integrates them into the corresponding nodes of the external graph, providing node-level representations. An external GNN learns host-level embeddings based on the external graph topology and the aggregated internal representations, enabling anomaly detection at the network level.

The overall forward pass of our IDS-NGNN framework is described in Algorithm 1, which explains how internal node embeddings are computed using a GAT layer, aggregated via the nested graph layer, and subsequently processed by an external GAT layer to provide node-level predictions. By integrating these components, IDS-NGNN can detect unexpected anomalies within the local activity of hosts and their interactions with other hosts in the network.

The system is end-to-end optimized, allowing the model to learn from internal and external graphs concurrently, thus improving detection accuracy in complex network environments.

A. IDS-NGNN Hierarchical Representation

IDS-NGNN employs a hierarchical GNN architecture at the base level, a GNN operates over rooted subgraphs (e.g., 1-hop ego-networks) centered around each host node w . These subgraphs are denoted as Gh_w^v , where v is a node in the subgraph rooted at w . The representation of node v at time step $t + 1$ is updated as follows:

$$h_{t+1}^{v, Gh_w^v} = U_t(h_t^{v, Gh_w^v}, m_{t+1}^{v, Gh_w^v}), \quad (1)$$

where h_{t+1}^{v, Gh_w^v} is the updated embedding of node v in subgraph Gh_w^v , $U_t(\cdot)$ is the node update function at iteration t , m_{t+1}^{v, Gh_w^v} is the aggregated message from the neighbors of v in Gh_w^v . The message aggregation is defined as:

$$m_{t+1}^{v, Gh_w^v} = \sum_{u \in N(v|Gh_w^v)} M_t(h_t^{v, Gh_w^v}, h_t^{u, Gh_w^v}, e_{vu}), \quad (2)$$

where $N(v|Gh_w^v)$ denotes the neighbors of node v within subgraph Gh_w^v , $M_t(\cdot)$ is the message function that aggregates features from v and its neighbor u , e_{vu} is the edge feature (e.g., communication type or frequency) between v and u .

Once node embeddings are obtained, a local pooling function $R_0(\cdot)$ aggregates them to form a single embedding for the subgraph (host-level representation):

$$h_G^w = R_0(\{h_T^{v, Gh_w^v} \mid v \in Gh_w^v\}), \quad (3)$$

where T is the total number of message-passing iterations, h_G^w is the pooled embedding of the host subgraph rooted at w .

At the global level, another GNN treats each h_G^w as a node in a higher-level graph G , representing the full network. A global pooling function $R_1(\cdot)$ computes the final graph representation:

$$h_G = R_1(\{h_G^w \mid w \in G\}), \quad (4)$$

where h_G encodes both intra-host activity and inter-host relationships.

This nested GNN design enables IDS-NGNN to hierarchically learn from fine-grained host-level behavior and high-level

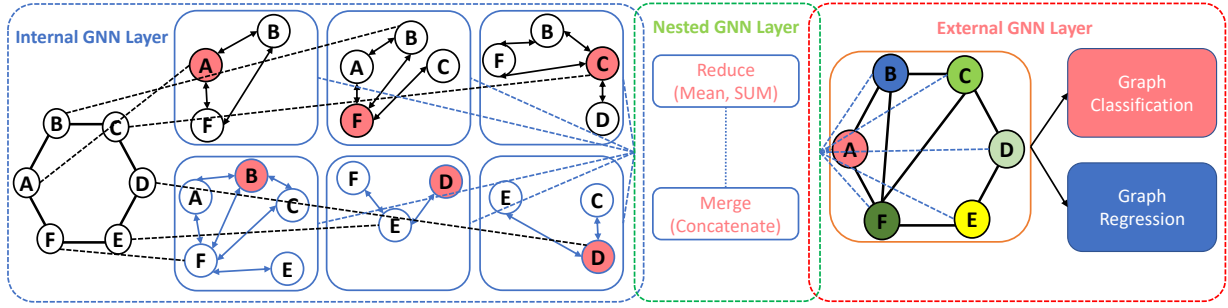


Fig. 1. Architecture of the IDS-NGNN model, composed of three layers. The blue rectangle (left) represents the internal layer, where node-level ego-nets are extracted. The green rectangle (center) shows the nested and merge layers, where local subgraphs are processed and embeddings are aggregated. The red rectangle (right) denotes the external GNN layer, which learns a global representation from the merged embeddings.

network interactions, improving robustness and expressivity in complex intrusion scenarios.

B. Internal Layer

Let $A_{in} \in \mathbb{R}^{N \times n \times n}$ and $H_{in} \in \mathbb{R}^{N \times n \times m}$ represent the adjacency matrices and node features for N internal graphs, each with at most n nodes and m -dimensional features.

The node features for the $(i - th)$ internal graph are:

$$h_i = \{\vec{h}_i^a, \vec{h}_i^b, \dots, \vec{h}_i^{n_i}\}, \quad \vec{h}_j \in \mathbb{R}^m, \quad (5)$$

where $n_i \leq n$. The internal GNN module computes updated embeddings for the i -th graph:

$$\{\vec{h}_i^{a'}, \dots, \vec{h}_i^{n_i'}\}' = GNN_{in}(h_i). \quad (6)$$

Assuming graph convolutional layers, a typical layer operation is:

$$H^{l+1} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^l W^l), \quad (7)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ is the degree matrix, W^l are learnable weights, and σ is an activation function like ReLU.

The internal GNN outputs $H'_{in} \in \mathbb{R}^{N \times n \times m'}$, where m' is the new feature dimension. All internal graphs share the same GNN architecture and parameters, enabling efficient and consistent learning.

C. Nested Graph Layer

An intermediate graph layer is placed between the internal and external graph layers, where the newly generated internal node embedding H'_{in} is used to update the external graph with the original external node feature vector H_{out} , as seen in Figure 1. $H_{out} = \{\vec{H}_1, \vec{H}_2, \vec{H}_3, \dots, \vec{H}_N\}$, $\vec{H}_l \in \mathbb{R}^m$ where M denotes the number of features of each external node. The *reduce* and *merge* operations are carried out within the stacked graph layer. Here, *reduce*() denotes the reduction function (e.g., mean, sum), and Equation 8 illustrates how the *reduce* operation generates an embedding for the $(i - th)$ internal network.

$$\vec{h}_i' = reduce(\vec{h}_i^{a'}, \vec{h}_i^{b'}, \dots, \vec{h}_i^{n_i'}) \quad (8)$$

The merge operation combines the original features of the external nodes with the newly created features from the internal network, as produced by the stacked graph layer.

To do this, Equation 9 shows how to define the updated external node feature $\vec{H}_i'$, where *merge*() stands for the merge function, which in our current design is the concatenation operation.

$$\vec{H}_i' = merge(\vec{H}_i, \vec{h}_i^{n'}) \quad (9)$$

Equation 10 shows the construction of the $N(\cdot)$ layer of the layered graph at the $(i - th)$ external node.

$$N(\vec{H}_1', \vec{h}_1^{a'}, \dots, \vec{h}_1^{n_i'}) = merge(\vec{H}_1', reduce(\vec{h}_1^{a'}, \vec{h}_1^{b'}, \dots, \vec{h}_1^{n_i'})) \quad (10)$$

D. External GNN Layer

At this stage, the nested graph has been fully constructed with attributed nodes. The next step is to apply the external GNN module, as illustrated in Figure 1. The final external node embedding set H''_{out} is produced by the external GNN module, which is used as input for the updated external graph feature matrix.

Such as $H''_i = \{\vec{H}_1'', \vec{H}_2'', \dots, \vec{H}_N''\}$ and the original external graph adjacency matrix A_{out} .

For all external nodes, the output node embedding set H''_{out} is determined by Equation 11 using the $GNN_{out}(\cdot)$ layer as the externalmost layer of the network. The output node embedding size M' is represented by the dimensions of the vector H''_{out} , which are $[N, M']$.

$$\{\vec{H}_1'', \dots, \vec{H}_N''\} = GNN_{out}\{\vec{H}_1', \dots, \vec{H}_N'\} \quad (11)$$

Mathematically, this is expressed as Equation 7 if we use graph convolution for the external layers as well.

We perform an embedding analysis, as described in Algorithm 2, by partitioning internal node embeddings according to their labels and computing class-wise mean vectors to distinguish benign from compromised hosts.

Algorithm 2 Embedding Analysis for Benign vs. Compromised

Require: Internal embeddings $I = \{\mathbf{h}_i\}_{i=1}^n$, node labels $y \in \{0, 1\}^n$

- 1: $M_{\text{benign}} \leftarrow \{i \mid y_i = 0\}$
- 2: $M_{\text{compromised}} \leftarrow \{i \mid y_i = 1\}$
- 3: $\mu_{\text{benign}} \leftarrow \frac{1}{|M_{\text{benign}}|} \sum_{i \in M_{\text{benign}}} \mathbf{h}_i$
- 4: $\mu_{\text{comp}} \leftarrow \frac{1}{|M_{\text{compromised}}|} \sum_{i \in M_{\text{compromised}}} \mathbf{h}_i$
- 5: **return** $\mu_{\text{benign}}, \mu_{\text{comp}}$

III. IDS-NGNN IMPLEMENTATION

The goal of compromised host detection in corporate networks is to identify infected machines exhibiting malicious behaviors such as lateral movement [24], data exfiltration [25], and malware activity [26]. Our approach leverages a nested and attributed graph structure to model internal host behavior and inter-host communication. Table I summarizes the graph statistics. The dataset comprises 336 unique hosts connected by 206,839 edges, with each node described by 84 flow-based features. The average in-degree is 3.85, reflecting moderate levels of network interaction. At the internal level, the graph contains 4,598 nodes and 5,120 edges, while the external representation consists of 336 hosts connected by 1,450 links.

We implement IDS-NGNN in PyTorch (v2.4) with PyTorch Geometric (PyG) for the GNN layers, and employ PyTorch Distributed Data Parallel (DDP) for multi-GPU training across two server testbeds, as detailed in Section IV. To mitigate overfitting and improve training efficiency, we apply per-graph feature normalization and prune ultra-sparse input features (e.g., non-zero ratio $< 2\% - 5\%$).

In our model, the base encoders are 2-layer **GCN**, **GraphSAGE**, or **GAT** (with 4 heads), each configured with a hidden size of $H=128$ and a dropout rate of 0.5. The nested block supports two pooling strategies: mean pooling and lightweight global attention pooling, followed by a *merge* operation (SUM or CONCAT) with node embeddings and a final graph convolution. Training is performed for 15–100 epochs, depending on workload size, as described in Section IV, using the Adam optimizer (learning rate 10^{-3}), weight decay in the range $10^{-4} - 5 \times 10^{-4}$, and batch sizes between 256–512.

A. Conventional GNN-based IDS vs IDS-NGNN-based IDS

Traditional GNN-based IDS solutions often rely on manually defined features to extract useful information from traffic traces. Prior studies [27] highlight two broad categories: *stateful* attributes (e.g., connection duration, packet counts, start/end times) and *stateless* attributes (e.g., source/destination IP addresses, port numbers, packet lengths). Stateful features capture the temporal dynamics of sessions, while stateless

TABLE I
GRAPH SPECIFICATIONS FOR COMPROMISED HOST DETECTION IN NESTED AND ATTRIBUTED GRAPHS

Parameter	Value
Compromised Host Detection Graph	
Graph Out V (Outgoing Nodes)	336
Graph Out E (Outgoing Edges)	206839
Graph Out H (Features)	84
Avg. In V (Average Incoming Nodes)	3.85
Avg. In H (Average Incoming Features)	84
# Class (Attack vs. Benign)	2 (binary) / 11 (multi-class)
Nested Graph Representation	
Nested Graph V (Internal Nodes)	4598
Nested Graph E (Internal Edges)	5120
Nested Graph H (Features)	84
External Graph V (Global Hosts)	336
External Graph E (Inter-Host Links)	1450
Attributed Graph Representation	
Attributed Graph V (Nodes)	4598
Attributed Graph E (Edges)	206839
Attributed Graph H (Features)	84

features describe the static properties of packets. Both are valuable for anomaly detection, but their reliance on manual feature selection limits generalization to unseen attack patterns.

In our framework, each host is represented as a node, and edges capture communication and authentication events (e.g., TCP/UDP flow, login sessions, SSH/RDP connections). These edges encode both traffic dynamics and trust relationships among hosts, enriching the attributed host graph with temporal and structural information essential for intrusion detection. Conventional models such as GCN, GraphSAGE, and GAT operate directly on this graph but remain limited in capturing long-range dependencies. However, IDS-NGNN introduces a nested architecture: each host is internally modeled by a process-alive subgraph (where nodes represent processes and edges capture co-activity lasting at least 1 s). Internal embeddings are then propagated to the external host graph, enabling hierarchical reasoning across both local and global contexts. This design eliminates the need for handcrafted attributes and improves robustness against structural perturbations.

IV. EXPERIMENTS AND ANALYSIS

For the experiments, we utilized two Dell R760 servers, each equipped with an Intel® Xeon® Gold 5418Y processor, and a SmartNIC Converged Accelerated card with NVIDIA A30x GPU. The servers were connected via high-speed 100 Gbps Ethernet and Ubuntu 22.04 as the operating system. The multi-GPU training was performed using PyTorch’s DDP, which synchronized the model’s weights and gradients across the two GPUs using NVIDIA flare library. Each GPU processed a distinct subset of the data in parallel, reducing training time and enabling faster convergence on large-scale datasets. Training, validation, and testing are performed at a ratio of 1:1:3. Each experiment is repeated 10 times, and the mean is published. We classify models according to their precision, macro F1 score,

TABLE II
 ACCURACY (%) ACROSS FIVE BENCHMARK IDS DATASETS. VALUES ARE MEAN $\pm$ STD.

Dataset	#Graphs	Avg. #nodes	GCN		GraphSAGE [18]		GAT	
			Base	Nested	Base	Nested	Base	Nested
CIC-IDS-2017	1113	39.1	92.0 $\pm$ 0.1	93.0 $\pm$ 0.1	94.0 $\pm$ 0.1	94.0 $\pm$ 0.1	95.0 $\pm$ 0.3	95.0 $\pm$ 0.3
CSE-CIC-IDS-2018	344	14.3	56.4 $\pm$ 7.1	57.0 $\pm$ 7.3	57.0 $\pm$ 5.5	56.7 $\pm$ 8.1	64.9 $\pm$ 6.3	65.1 $\pm$ 7.2
ToN-IoT	500	25.0	60.0 $\pm$ 4.0	61.5 $\pm$ 5.0	58.0 $\pm$ 3.5	59.0 $\pm$ 4.0	62.5 $\pm$ 5.0	63.0 $\pm$ 4.5
CICIoT2023	750	30.0	65.0 $\pm$ 3.5	66.0 $\pm$ 3.0	64.5 $\pm$ 4.0	65.5 $\pm$ 3.5	67.0 $\pm$ 3.0	67.5 $\pm$ 4.0
CICIoV2024	1000	50.0	70.0 $\pm$ 4.5	71.5 $\pm$ 4.0	69.0 $\pm$ 5.0	70.5 $\pm$ 4.5	72.0 $\pm$ 4.0	73.0 $\pm$ 4.5

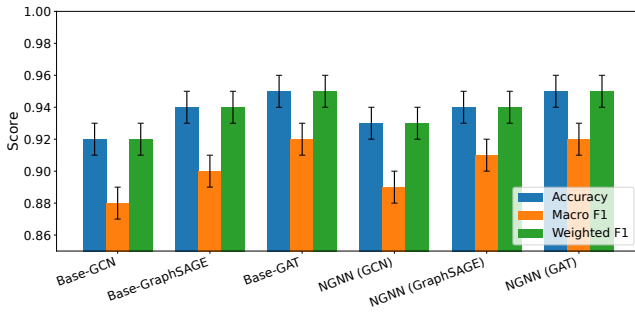


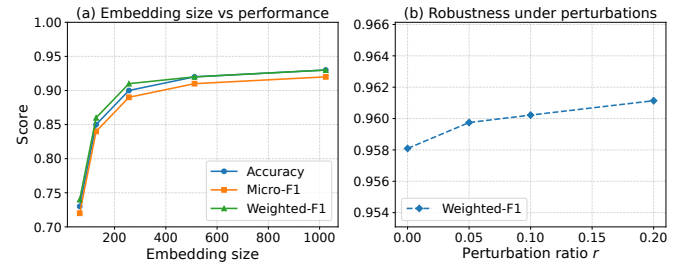
Fig. 2. Aggregated performance results of each model.

and weighted F1 score. The percentage of occurrences that are appropriately labeled is used to calculate accuracy.

A. Performance Analysis

A summary of the dataset statistics is presented in Table II, which includes results from five benchmark IDS datasets. On CIC-IDS-2017, Base-GCN achieves 92% accuracy, while IDS-NGNN(GAT) increases this to 95%. Similarly, Base-GraphSAGE reaches 94%, and Base-GAT 95%, with NGNN variants sustaining consistent improvements. On newer IoT-oriented datasets, IDS-NGNN continues to provide small but stable gains, confirming that nested reasoning generalizes across domains. We contrast Base-Graph Convolutional Networks (GCN), Base-GraphSAGE, and Base-GAT, three widely used GNN techniques. On the nested graph, we evaluated three different IDS-NGNN variants by employing GCN, GraphSAGE, or GAT as the internal and external layers within the architecture. IDS-NGNN improves the aggregated accuracy to 95%, the macro F1 score to 92%, and the weighted F1 score to 92%, as shown in Figure 2.

In terms of accuracy and F1 metrics, this corresponds to a $\sim 3\text{--}4\%$ gain over the strongest baseline. All IDS-NGNN configurations also exhibit consistent macro F1 improvements over their base models, demonstrating that the learned hierarchical embeddings effectively capture both host-level and network-level dependencies. The performance difference between the NGNN variants is modest, with the best results obtained using GAT for both the internal and external layers. Leveraging attention enables GAT to focus on the most critical nodes and edges, resulting in superior detection performance.


 Fig. 3. IDS-NGNN performance analysis. (a) Accuracy, Micro-F1, and Weighted-F1 across embedding sizes. (b) Structural robustness under edge perturbations r , showing less than 2% degradation up to $r = 0.20$.

For comparison, the manually generated feature-based baseline (Base-GCN) achieves a weighted F1 of only 74%. In contrast, IDS-NGNN without explicit host attributes achieves 83% weighted F1, suggesting that the process-alive graph (internal graph) already encodes sufficient process host behavioral characteristics for intrusion detection.

The performance of $N(\text{GCN}, \text{GCN})$ across various embedding sizes ranging from 64 to 1024 is illustrated in Figure 3(a). IDS-NGNN maintains consistent performance across all three metrics despite changes in embedding size. Accuracy improves from 73% at size 64 to 93% at size 1024, confirming the effectiveness of the learned embeddings. Even small embeddings deliver competitive performance, highlighting scalability to resource-constrained settings. Figure 3(b) assesses robustness against structural perturbations: The Weighted-F1 score decreases slightly, from 0.958 at $r = 0.00$ to approximately 0.954 at $r = 0.20$, representing a relative reduction of less than 2%. This demonstrates resilience to adversarial edge insertions and deletions, as well as stability under graph manipulations.

B. Complexity Analysis

We next analyze computational complexity and runtime efficiency. Table III reports FLOPs per forward pass, where IDS-NGNN adds only $+50\text{--}80\%$ overhead compared to flat GNNs, keeping all models under 10 MFLOPs per graph. This modest increase justifies the three-part design, which expands receptive fields without heavy cost.

To complement FLOPs, Table IV reports wall-clock preprocessing, training, and inference times on Xeon CPU, NVIDIA A30 GPU, and SmartNIC+GPU (A100) pipelines. On the 6k-graph workload, GPU training is $\sim 2.4\times$ faster than CPU (124.9s $\rightarrow$ 51.7s), while SmartNIC+GPU reduces training

further to 48.6s. Inference latency remains sub-millisecond, dropping from 0.402 ms/graph on GPU to 0.352 ms/graph with SmartNIC integration. FLOPs scale with N , E , and H : larger/denser 6k graphs explain higher preprocessing time and inference latency. GPU acceleration is most evident in models with deeper and attention-heavy architectures (e.g., GAT); short, shallow runs underutilize the GPU. Despite scale-up, IDS-NGNN maintains sub-ms inference per graph, confirming suitability for real-time 100 Gbps intrusion detection.

TABLE III

PER-FORWARD FLOPs (MFLOPs) ON CICIDS2017 (AVG. GRAPH $N = 20$, $E = 82$, $F = 84$, $H = 128$, $A = 4$ HEADS).

Model	2-Layer	+Nested (SUM/CONCAT)
GCN	1.24 M	1.94 M / 2.25 M
GraphSAGE	1.22 M	1.91 M / 2.22 M
GAT (4 heads)	4.96 M	7.69 M / 8.98 M

TABLE IV

RUNTIME BENCHMARKS ON CICIDS2017 (6,000 GRAPHS, BATCH SIZE 512, GAT, 15 EPOCHS).

Setting	Preprocess	Training	Inference
CPU-only (Xeon)	320.8 ms	124.9 s	—
GPU (A30)	318.3 ms	51.7 s	0.402 ms
SmartNIC+GPU (A100) *	≤325.6 ms	48.6 s	0.352 ms

V. CONCLUSION

In this paper, we presented IDS-NGNN, a nested graph neural network architecture for intrusion detection that jointly captures local host-level and global network-level dependencies. Evaluations on five public datasets showed up to 3.5% accuracy improvement over baselines, with the N (GAT, GAT) variant achieving 95% accuracy and 0.92 macro-F1. Complexity analysis confirmed that nested blocks add only modest overhead (< 10 MFLOPs/graph), while hardware benchmarks demonstrated sub-millisecond inference latency with SmartNIC+GPU acceleration. These results establish IDS-NGNN as both accurate and computationally efficient for real-time 100 Gbps intrusion detection. Future work will extend the framework to multi-tenant environments and explore resilience against adaptive adversarial attacks.

ACKNOWLEDGMENT

This work is supported by the European Union's Horizon RIA research and innovation programme under grant agreement No. 101092908 (SMARTEDGE).

REFERENCES

- [1] J. Firch, Cyber Security Statistics: The Ultimate List Of Stats, Data & Trends, Retrieved September 10 (2021) 2021.
- [2] R. Abu Bakar, X. Huang, M. S. Javed, S. Hussain, M. F. Majeed, An Intelligent Agent-Based Detection System for DDoS Attacks Using Automatic Feature Extraction and Selection, *Sensors* 23 (6) (2023) 3333.
- [3] Cloudflare, Ddos threat report for 2024 q4, <https://shorturl.at/jw6kr>, accessed: [07-02-2025] (2024).

- [4] O. Yoachimik, J. Pacheco, Cloudflare's 2025 q1 ddos threat report, <https://shorturl.at/1prSA>, accessed: 2025-04-28 (2025).
- [5] W. Turton, K. Mehrotra, Hackers breached colonial pipeline using compromised password, *Bloomberg.com* 4 (2021).
- [6] C. Rajathi, P. Rukmani, Hybrid learning model for intrusion detection system: A combination of parametric and non-parametric classifiers, *Alexandria Engineering Journal* 112 (2025) 384–396.
- [7] X. Guan, S. Zhu, X. Wan, Y. Wu, Starnet: Multidimensional spatial-temporal attention recall network for accurate encrypted traffic classification, *Journal of Network and Computer Applications* (2025) 104109.
- [8] P. Castoldi, R. A. Bakar, A. Sgambelluri, J. J. V. Olmos, F. Paolucci, F. Cugini, Programmable packet-optical network security and monitoring using dpus with embedded gpus, *Journal of Optical Communications and Networking* 17 (2) (2025) A178–A195.
- [9] T. Das, R. M. Shukla, S. Sengupta, Poisoning the well: Adversarial poisoning on ml-based software-defined network intrusion detection systems, *IEEE Transactions on Network Science and Engineering* (2024).
- [10] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, R. M. Parizi, A robust privacy-preserving federated learning model against model poisoning attacks, *IEEE Transactions on Information Forensics and Security* (2024).
- [11] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, *IEEE Communications Surveys & Tutorials* 25 (1) (2023) 538–566.
- [12] T. Bilot, N. El Madhoun, K. Al Agha, A. Zouaoui, A survey on malware detection with graph representation learning, *ACM Computing Surveys* 56 (11) (2024) 1–36.
- [13] A. Khraisat, A. Alazab, S. Singh, T. Jan, A. Jr. Gomez, Survey on federated learning for intrusion detection system: Concept, architectures, aggregation strategies, challenges, and future directions, *ACM Computing Surveys* 57 (1) (2024) 1–38.
- [14] M. Faheem, M. A. Al-Khasawneh, Multilayer cyberattacks identification and classification using machine learning in internet of blockchain (iobc)-based energy networks, *Data in Brief* 54 (2024) 110461.
- [15] G. George, S. M. Thampi, A graph-based security framework for securing industrial iot networks from vulnerability exploitations, *IEEE Access* 6 (2018) 43586–43601.
- [16] M. Zhong, M. Lin, C. Zhang, Z. Xu, A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges, *Computers & Security* (2024) 103821.
- [17] M. Zhang, P. Li, Nested graph neural networks, *Advances in Neural Information Processing Systems* 34 (2021) 15734–15747.
- [18] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-graphsage: A graph neural network based intrusion detection system for iot, in: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, IEEE, 2022, pp. 1–9.
- [19] L. Van Langendonck, I. Castell-Uroz, P. Barlet-Ros, Ppt-gnn: A practical pre-trained spatio-temporal graph neural network for network security, *arXiv preprint arXiv:2406.13365* (2024).
- [20] D.-H. Tran, M. Park, Fn-gnn: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems, *Applied Sciences* 14 (16) (2024) 6932.
- [21] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32, 2018.
- [22] K. Oono, T. Suzuki, Graph neural networks exponentially lose expressive power for node classification, *arXiv preprint arXiv:1905.10947* (2019).
- [23] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 2847–2856.
- [24] M. Pérez-Gomariz, F. Cerdan-Cartagena, J. García, Lm-hunter: An nlp-powered graph method for detecting adversary lateral movements in apt cyber-attacks at scale, *Computer Networks* 262 (2025) 111181.
- [25] F. Ullah, M. Edwards, R. Ramdhany, R. Chitchyan, M. A. Babar, A. Rashid, Data infiltration: A review of external attack vectors and countermeasures, *Journal of Network and Computer Applications* 101 (2018) 18–54.
- [26] J. Ispahany, M. R. Islam, M. Z. Islam, M. A. Khan, Ransomware detection using machine learning: A review, research limitations and future directions, *IEEE Access* 12 (2024) 68785–68813.
- [27] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, Neural message passing for quantum chemistry, in: *International conference on machine learning*, PMLR, 2017, pp. 1263–1272.